

# 15. FastAPI Basics

Phase: K

**Code:** `search_api/main.py`, `search_api/routers/health.py`, `search_api/models/responses.py`, `tests/search_api/test_health.py`

## What it is

FastAPI is a **web framework** — a library that does the repetitive, boring plumbing of "listen for an incoming web request, figure out which piece of your code should handle it, run that code, and send a response back" so you only have to write the part that's actually specific to your project.

A few pieces of jargon, unpacked:

- **App instance** — a single Python object (created once, when the program starts) that represents "the whole web application." Every URL your service responds to, every setting, every piece of documentation is attached to this one object. When you run the server, you're really telling it "start listening for requests and hand them to *this* object."
- **Endpoint** — one specific URL + HTTP method combination your app can respond to, e.g. "a GET request to `/health`." Each endpoint is backed by an ordinary Python function; FastAPI's job is to notice which function to call for which incoming request.
- **Router** — a way of grouping related endpoints into their own file instead of dumping every endpoint into one giant file. A router isn't runnable on its own; it gets *attached* to the app instance before the app can actually serve those endpoints.
- **Request/response validation (Pydantic models)** — before your function even runs, and again after it returns a value, FastAPI checks the shape of the data against a schema you defined ahead of time (a Python class listing exactly which fields exist and what type each one is). If your function tries to return something that doesn't match the schema, FastAPI raises an error instead of silently sending broken data to whoever asked for it.
- **TestClient** — a tool that lets you call your endpoints directly in a test file, in-process, without starting a real server on a real network port. It behaves like a web browser or another program calling your API, but everything happens inside the same Python process your test is running in, so tests are fast and don't need a live deployment.

## Real-life analogy

Think of a company's front desk / reception area.

- The **building itself** (with its address, its front doors, its directory sign) is the **app instance** — one single thing that represents "this company is open for business here."

- Each **department** (HR, Payroll, IT Support) is a **router** — a self-contained group of services that live in their own wing, with their own staff, filed under their own name in the directory. HR doesn't need to know how Payroll processes a request; each department is written and organized independently, then listed in the building's directory so visitors can find it.
- A **specific service window**, like "Submit a vacation request at the HR desk," is an **endpoint** — one exact thing you can ask for, at one exact location.
- The **intake form** that HR requires before they'll even look at your request — first name, last name, dates, must all be filled in the right format — is **request validation**. If you hand them a form with the "dates" field containing your favorite color, they hand it back before doing any work at all.
- The **standardized receipt** they hand you back — always has a confirmation number, a status, and a timestamp, never just a random scrap of paper — is **response validation**. Whoever built the receipt template decided in advance exactly what fields a receipt must contain, and the front desk staff can't accidentally forget one.
- **TestClient** is a company auditor who walks up to any service window and asks a question, then checks the answer — without needing the whole building to be open to the public, without a real visitor showing up, and without leaving the building at all.

## Why we used it here

Requirement **R18** calls for a `GET /health` endpoint that reports whether the Search API's dependencies (the database, the pgvector indexes, the BM25 indexes) are actually reachable and correctly set up — the kind of endpoint a deployment tool or monitoring dashboard pings to decide "is this service safe to route traffic to right now?"

FastAPI was already the client-mandated framework for this whole project (see `CLAUDE.md`: "client-preferred; async, OpenAPI docs generation satisfies the Swagger deliverable"), so `/health` is built the same way every future endpoint in `search_api/` will be:

- `search_api/main.py` creates the **one** app instance for the whole service and is the file `uvicorn` actually starts.
- `search_api/routers/health.py` is a self-contained **router** — the health check's URL, its logic, and its logging all live in one file that doesn't know or care about `/search/category` or `/match/companies`, which will live in their own router files later.
- `search_api/models/responses.py` defines `HealthResponse` and `HealthChecks` as **Pydantic models** so that `/health` always returns the exact same shape — `status`, `checks`, `response_time_ms` — never an ad-hoc dictionary that might be missing a field on some code path and not others.
- `tests/search_api/test_health.py` uses **TestClient** to prove the endpoint behaves correctly in both the healthy case and the degraded case (simulated DB outage), without needing a real Postgres-backed server running somewhere during the test run.

## Code walkthrough

## 1. The FastAPI app instance — search\_api/main.py

```
from fastapi import FastAPI

from search_api.routers import health

# Create the ONE app instance for the whole service. Everything else –
# every router, every setting – gets attached to this single object.
app = FastAPI(
    title="Bilingual Search API",          # shows up in the auto-generated docs page
    description="Hybrid (BM25 + vector, RRF-fused) search over bilingual category/company data.",
)

# Attach the health router's endpoints to the app. Without this line,
# /health would be fully written and correct but completely unreachable –
# the app wouldn't know it exists.
app.include_router(health.router)
```

This file is intentionally tiny. It doesn't contain any endpoint logic itself — it only creates the app and wires routers into it. `uvicorn search_api.main:app --reload --port 8000` (from CLAUDE.md's Build/Test/Run section) tells uvicorn (the actual program that listens on a network port) to import this file and hand incoming requests to the app object defined in it.

## 2. APIRouter for modular endpoint organization — search\_api/routers/health.py

```
from fastapi import APIRouter
from loguru import logger

from search_api.db import get_connection
from search_api.models.responses import HealthChecks, HealthResponse

# A router is a mini, self-contained app. It can define endpoints on its
# own, but it does nothing on its own – it has to be included into the
# real `app` (see main.py's app.include_router(health.router)) before any
# of its endpoints are actually reachable.
router = APIRouter()

REQUIRED_INDEXES = {
    "idx_category_vectors_embedding",
    "idx_company_vectors_embedding",
    "idx_category_bm25_hu",
    "idx_category_bm25_en",
    "idx_company_bm25_hu",
    "idx_company_bm25_en",
}

def _check_indexes(conn) -> tuple[bool, bool]:
    # Plain helper function, no FastAPI involved – checks that both the
    # vector indexes and the BM25 indexes exist in the database.
    with conn.cursor() as cur:
        cur.execute(
            "SELECT indexname FROM pg_indexes WHERE indexname = ANY(%s)",
            (list(REQUIRED_INDEXES),),
        )
        found = {row[0] for row in cur.fetchall()}
```

```

vector_index = {"idx_category_vectors_embedding", "idx_company_vectors_embedding"} <= found
bm25_index = {
    "idx_category_bm25_hu",
    "idx_category_bm25_en",
    "idx_company_bm25_hu",
    "idx_company_bm25_en",
} <= found
return vector_index, bm25_index

# @router.get("/health", ...) is a decorator: it tells FastAPI "whenever a
# GET request arrives for the URL /health, call the function directly
# below this line and use whatever it returns as the response."
# response_model=HealthResponse is the validation contract – see section 3.
@router.get("/health", response_model=HealthResponse)
def health() -> HealthResponse:
    start = time.perf_counter()

    database = vector_index = bm25_index = False
    try:
        conn = get_connection()
        try:
            with conn.cursor() as cur:
                cur.execute("SELECT 1") # simplest possible "is the DB alive?" query
                database = True
                vector_index, bm25_index = _check_indexes(conn)
        finally:
            conn.close()
    except Exception as e:
        # DB unreachable: don't crash the endpoint, report it as "degraded" instead.
        logger.warning(f"EVIDENCE_API_HEALTH_DB_UNREACHABLE: {e}")

    all_ok = database and vector_index and bm25_index
    response_time_ms = (time.perf_counter() - start) * 1000

    logger.info(
        f"EVIDENCE_API_HEALTH_CHECK: status={'ok' if all_ok else 'degraded'} "
        f"database={database} vector_index={vector_index} bm25_index={bm25_index}"
    )

    # Return a HealthResponse instance, not a raw dict. FastAPI converts it
    # to JSON for the actual HTTP response.
    return HealthResponse(
        status="ok" if all_ok else "degraded",
        checks=HealthChecks(database=database, vector_index=vector_index, bm25_index=bm25_index),
        response_time_ms=response_time_ms,
    )

```

Notice the function is named `health()` just like the endpoint — nothing FastAPI-specific about the name, it's an ordinary Python function. What makes it an endpoint is purely the `@router.get("/health", ...)` decorator sitting above it. Also notice the deliberate design choice, called out in this file's own module docstring: a missing index degrades status to "degraded" rather than raising an exception that would turn into an HTTP 500 error — a monitoring tool checking `/health` gets a real, structured answer either way, rather than the endpoint itself falling over.

### 3. Pydantic response\_model validation — `search_api/models/responses.py`

```

from typing import Literal

```

```

from pydantic import BaseModel, Field

class HealthChecks(BaseModel):
    # Each attribute here is a field FastAPI (via Pydantic) will require
    # and validate. `bool` means: this must be a real True/False value –
    # not "true", not 1, not missing.
    database: bool
    vector_index: bool
    bm25_index: bool

class HealthResponse(BaseModel):
    # Literal["ok", "degraded"] is stricter than `str` – it says this field
    # may ONLY ever be one of these two exact values. If the endpoint code
    # tried to return status="fine", Pydantic would reject it before the
    # response ever went out.
    status: Literal["ok", "degraded"]

    # A nested Pydantic model – the response's `checks` field is itself a
    # HealthChecks object, so the JSON comes out as a nested object:
    # {"status": "ok", "checks": {"database": true, ...}, "response_time_ms": 4.2}
    checks: HealthChecks

    # Field(..., description=...) both marks this field as required (the
    # `...` means "no default, must always be provided") and attaches a
    # human-readable description that shows up in the auto-generated API
    # docs page.
    response_time_ms: float = Field(..., description="Total time to run all health checks")

```

The connection back to `health.py` is the `response_model=HealthResponse` argument on the `@router.get(...)` decorator. That one keyword argument is what makes FastAPI check, after `health()` returns, "does this returned object actually match `HealthResponse`'s shape?" If a future edit to `health()` accidentally forgot to set `response_time_ms`, or set `status` to some new typo'd string, FastAPI would raise a server error right there rather than silently sending malformed JSON to whoever called the API — this is the "standardized receipt" from the front-desk analogy: the shape of the response is guaranteed, not just hoped for.

#### 4. *TestClient for testing without a running server* — `tests/search_api/test_health.py`

```

from fastapi.testclient import TestClient

from search_api.main import app

# Wrap the real app instance in a TestClient. This does NOT start uvicorn
# or open a real network port – it calls straight into the app's code,
# in-process, the same way a browser's request would arrive, but entirely
# inside this test run.
client = TestClient(app)

def test_health_ok_against_real_db():
    response = client.get("/health")          # behaves exactly like a real HTTP GET
    assert response.status_code == 200
    body = response.json()                   # parse the JSON body, same as any HTTP client would
    assert body["status"] == "ok"
    assert body["checks"] == {"database": True, "vector_index": True, "bm25_index": True}

```

```

assert body["response_time_ms"] > 0

def test_health_degrades_when_db_unreachable(monkeypatch):
    def broken_connection():
        raise ConnectionError("simulated outage")

    # Swap out get_connection() inside the health router, for the duration
    # of this one test, so it always fails – simulating a real DB outage
    # without needing to actually take a database down.
    monkeypatch.setattr("search_api.routers.health.get_connection", broken_connection)

    response = client.get("/health")
    assert response.status_code == 200          # still 200, not 500 – see health.py's try/except
    body = response.json()
    assert body["status"] == "degraded"
    assert body["checks"] == {"database": False, "vector_index": False, "bm25_index": False}

```

Because `TestClient` calls into the real app object — the same one `uvicorn` would serve in production — these tests exercise the actual routing (`app.include_router`), the actual endpoint function, and the actual Pydantic validation, all without a real network connection or a separately running server process. That's what "without a running server" means here: no `uvicorn` process, no open port, no real HTTP traffic over a socket — just direct, in-process function calls dressed up to look and behave like HTTP.

### ***Coming later:*** `Depends()` *and dependency injection*

None of the code above uses FastAPI's `Depends()` yet, but it's worth previewing because it will show up soon (auth checks, database session handling for the search endpoints).

The idea: instead of an endpoint function reaching out and creating things it needs (like `get_connection()` does today, called directly inside `health()`), you instead declare "this endpoint *needs* a database connection" as a parameter, and hand FastAPI a separate function that knows how to produce one. FastAPI calls that separate function for you, before your endpoint runs, and hands you the result as an argument.

Using the front-desk analogy: today, the HR desk clerk personally walks to the supply closet, gets a stapler, and staples your form. With dependency injection, the clerk instead says "I need a stapler" and someone else (a well-defined process, `Depends()`) is responsible for making sure a stapler shows up in their hand — the clerk doesn't need to know where the closet is, and if the process for getting a stapler changes later (a new supplier, a different closet), the clerk's job doesn't change at all.

Why this matters for what's coming: once `/search/category`, `/match/companies`, and other endpoints need a database connection *and* (per `.claude/rules/api-conventions.md`, no auth in v1, but this will change) potentially an authenticated user, repeating "manually open a connection, remember to close it, manually check a token" inside every single endpoint function is exactly the kind of repeated, easy-to-get-wrong plumbing `Depends()` exists to centralize — write the "how to get a DB connection" or "how to verify this request is authenticated" logic exactly once, then have every endpoint that needs it simply declare the dependency as a parameter.

## How to explain this in an interview/to a teammate

"FastAPI gives us one app instance that represents the whole Search API, and we split endpoints into routers so `/health` lives in its own file without needing to know anything about the search endpoints that will live alongside it. Every endpoint's response is defined as a Pydantic model up front — `HealthResponse` and `HealthChecks` here — so FastAPI validates the shape of what we return before it ever goes out as JSON; if the code tried to return something malformed, we'd get an error instead of a silently broken API. For testing, `TestClient` lets us call `/health` directly inside a test file, including simulating a DB outage with `monkeypatch`, without ever starting a real server or hitting a real network port. The one thing we haven't used yet is `Depends()` — dependency injection — but it's coming soon for auth and DB-session handling, so we don't repeat the same setup logic in every endpoint."